

Google SRE / SRE Manager — Interview Cheat Sheet

GSRE quiz bank (537 Q) · Your loop: Code review · Systems Architecture 60m · Googleness & Leadership 45m

① CODE REVIEW 45m

Google Doc · no IDE · CR + BS-IMPL

② SYSTEMS ARCH 60m

No coding · clarify · math · NALSD

③ G&L 45m

STAR · manager scenarios · conversational OK

Code review — triage order

1. **Correctness** — logic, off-by-one, null, races

2. **Concurrency / resources** — locks, leaks, OOM

3. **Security** — injection, pickle, secrets

4. **Reliability** — retries, timeouts, backpressure

5. **Observability** — metrics not print, HTTP codes

6. **Style / naming** — last, not first

Google Doc rules

Manual trace + propose test cases

Narrate aloud; ask when stuck

200-line CL: understand before fixing

Style nits are often **decoys**

Primitives

Lock = exclusive access

Semaphore(N) = cap concurrency at N

Always release in finally / context mgr

CR-6 production bugs (spot these)

Pattern	Fix instinct
readlines() huge file	Stream / chunk / seek
Fixed retry interval	Jitter + backoff + budget
Thread per request	Bounded pool / queue
Unbounded cache/queue	maxsize + shed load
No timeouts on HTTP/DB	connect + read deadlines
Retry w/o idempotency	Idempotency keys
Always HTTP 200	Propagate 4xx/5xx
user_id metric label	Bounded cardinality
SQL f-string / shell=True	Params / list args
pickle.loads untrusted	RCE — never
Busy-wait spin loop	Event.wait()
FD / connection leak	Context managers, pools
Blocking in async	Thread pool / rewrite

NALSD / Systems Architecture

Not greenfield — fix existing system

Clarify scope, scale, constraints first (~20 min)

BOM: machines, cores, disk, bandwidth

Iterate: possible → feasible → resilient

Incremental ops fixes > re-architect

When overloaded

Admission control · load shed · bounded queues

Return 503 early vs deep queue timeout

Fix **tail latency** (p99), not just average

Four golden signals

Latency · Traffic · Errors · Saturation

SLO / error budget

Agree SLIs before boxes

Exhausted budget → negotiate with data

Napkin math

Calc	Formula
Required cores	$QPS \times latency(s)$
Queue RAM	$depth \times payload\ size$
Storage/day	$writes/s \times bytes \times 86400 \times retention \times repl$
2^{32}	$\approx 4B\ (uint32\ max)$
Mid+high overflow	$low + (high - low) // 2$

Physics traps — say impossible = pass

5 PB / 10 Gbps ≈ **weeks**, not 4h RTO

1 TB → 10k nodes: tree/P2P fanout, not serial

78% avg CPU → bursts → queue → timeouts at peak

Troubleshooting — first moves

Impact — who, how much, since when?

Changes — deploys, flags, config, traffic

Mitigate before root cause (one change at a time)

Golden signals — don't trust CPU alone

Shakespeare blackbox alert

Probe red ≠ always user impact — verify real traffic

Check pipeline: index → serve → deps

Synthetic green, users red

Probe misses path / region / cohort

Compare canary coverage vs prod traffic

Post-deploy latency spike

Correlate timeline · canary vs full · rollback candidate

Linux & networking (E3)

Symptom	Check
Ping OK, HTTPS fail	L4–L7: ports, TLS, LB
IP works, hostname fails	DNS
TLS hang mid-handshake	Path MTU black hole
D-state process	Kernel I/O block
OOMKilled, host has RAM	cgroup limit
Connection failures at scale	Ephemeral ports / TIME_WAIT
futex storm in strace	Lock contention
Disk full, df shows space	Inodes

Phone-screen classics

Zombie = child exited, parent didn't wait()

Hard link = same inode; symlink = path pointer

ls -l: shell → fork → exec → getdents/stat

TCP reliable ordered; UDP fast, may drop

Weird & unexpected (WX) — don't get surprised

Loop / format surprises

Recruiter prep (mutex/FS) ≠ onsite shape — topics hide in review/NALSD

Manager loop still technical: code review + NALSD + G&L

Only 3 onsite rounds possible — confirm with recruiter

Must-pass code review fail → ~12 month cooldown (some reports)

SRE-SWE may get LeetCode; SRE-SE more practical — know your track

Walk away if role said troubleshooting but loop is only LeetCode

Apache live-debug gauntlet (Glassdoor)

① File permissions → check Apache logs

② File in different dir → read httpd.conf

③ Unicode homoglyph filename → search literal ASCII string

④ Segfault → gdb core dump → explain intentional SIGSEGV

Googleness & Leadership (C3, C5)

STAR: Situation → Task → Action → Result (+ Learning)

Prep: conflict with manager, critical feedback, teammate disagreement

25 min G&L can be fine — conversational rapport scores

High vs low performer: differentiated coaching, not identical treatment

Error budget exhausted: negotiate with data, don't just veto

Burnout: alert hygiene, fair rotation, reduce toil — not heroics

Sev-1 during demo: IC role, calm comms, mitigate first

Four attributes: GCA · RRR · Leadership · Googleness

IC coding awareness (not your loop, but reported)

tail large file: seek backward in chunks

Word frequency: stream once, hash map count

Confusing cities: count (city, name) pairs ≥ 3

Deadlock vs livelock: blocked forever vs busy, no progress

Systems Architecture — 60 min flow

0–20 min: Clarify — users, QPS, read/write ratio, latency, retention, consistency, failure tolerance

Then: High-level boxes + data flows (API, storage, cache, queue, CDN)

Deep-dive when asked: One component — APIs, keys, failure behavior

Always volunteer: How you know it's healthy (lag, drops, SLO alerts)

Vague prompts ("trends in unstructured data"): clarify data volume, consumers, latency, privacy, what "trend" means

Toy prompts ("snakes game"): still do requirements + math + SLOs + failure modes

Interviewer disengaged in NALSD: likely missed early clarifier — pause and restate scope